TEMA: Manejadores en Django

Resultados de Aprendizaje:

- Que los estudiantes comprendan como utilizar los principales manejadores, para poder realizar consultas a la base de datos

DESARROLLO

Cuando tabjamos con base de datos siempre surge la necesidad de poser realizar consultas desde muy sencilla como contar la cantidad de reguistros que existen para una determinada tabla , como poder realizar contultas relizando el filtado de datos por algun parametro , en esta guia aprenderemos como poder realizre este tipo de consultas utilizando los manejadores .

Requerimientos:

Para la realizacion de esta guia usted debe de contar con los siguientes elementos

- Contar con el proyecto comercial generado en la guia anterios
- Contar con un servidor de base de datos mysql

Manejadores en django

1. Los managers en Django son una herramienta poderosa que permite realizar diversas acciones dentro de las listas de objetos de un modelo, como contar, traer el primero o el último elemento, crear nuevos registros y mucho más.
2. En esta ocasión para poder probar los manejadores utilizaremos la consola interactiva , utilizanod el comando ❯ **python .\manage.py shell**
3. Una vez en la consola intetactiva importararemos los modelos categoira y producto par poder trabajr con ellos ,
   In [1]: from catalogo.models.producto import Producto
   In [2]: from catalogo.models.categoria import Categoria

Manager objects

4. Objects es el manejador por defecto de Django , este nos permite comunicarnos con la base de datos por medio de su ORM
5. Una de las primeras cosas que aprenderemos del manejador objects , es podre extraer todos los reguistro de un modelo , para ellos podemos utilizar el comando **> Categoria.objects.all() ,** este comando retornada un **QuerySet ,** con todos los reguistros asociados a dicha clase

```
In [1]: from catalogo.models.producto import Producto

In [2]: from catalogo.models.categoria import Categoria

In [3]: Categoria.objects.all()
Out[3]: <QuerySet [<Categoria: Categoria : Electrónicos Description: Productos e
lectrónicos Fecha de creación : 2026-09-14 18:43:41.575695+00:00 Fecha de actual
ización 2026-09-14 18:43:41.575724+00:00>, <Categoria: Categoria : Laptop Descri
ption: Computadora portatil Fecha de creación : 2026-09-14 21:29:19.524546+00:00
 Fecha de actualización 2026-09-14 21:29:19.524621+00:00>]>
```

El primer elemento

6. Si desearmos solo el primer elemento podremos oocupar el comando **> Categoira.objects.first() ,** denote que no esta mirando el id , esto es porque en la funcion __str__ no incorporamos el id , si hace las cambiso necesarios podra mostrar dicho elemento , pero tenga que encunta que debera de reinicar el shell

```
In [4]: Categoria.objects.first()
Out[4]: <Categoria: Categoría : Electrónicos Description: Productos electrónicos
 Fecha de creación : 2026-09-14 18:43:41.575695+00:00 Fecha de actualización 202
6-09-14 18:43:41.575724+00:00>
```

7. Modificando el __str__ del modelo categoira

```python
from django.db import models

# Create your models here.


class Categoria(models.Model):
    # define un campo de tipo texto con máximo de 255 caracteres
    nombre = models.TextField(max_length=255)
    # definite un campo de tipo texto que puede ser nulo con máximo de 255 caracteres
    descripcion = models.TextField(max_length=255, null=True)
    # define campos de tipo fecha y hora para almacenar la fecha de creación y actualización
    del registro
    created_at = models.DateTimeField()
    updated_at = models.DateTimeField()

    # function que mostrar la information de la tabla en consola
    def __str__(self):
        return f"id: {self.id} Nombre: {self.nombre} Description: {self.descripcion} Fecha de
        creación : {self.created_at} Fecha de actualización {self.updated_at}"
```

8. Modificando el __str__ de modelo producto

```python
from .categoria import Categoria


class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(max_length=255, null=True)
    # max_digits es el número máximo de dígitos que puede tener el número, incluyendo los decimales
    precio_compra = models.DecimalField(max_digits=10, decimal_places=2)
    precio_venta = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField()
    # relación de uno a muchos, un producto pertenece a una categoria y una categoria puede tener muchos productos
    categoria = models.ForeignKey(
        Categoria, on_delete=models.CASCADE, related_name='productos')
    # también lo puede ponder de esta manera
    # stock = models.IntegerField()
    # auto_now_add agrega la fecha de manera automática cuando se crea el objeto
    created_at = models.DateTimeField(auto_now_add=True)
    # auto_now actualiza la fecha de manera automática cuando se actualiza el objeto
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"id: {self.id} Nombre: {self.nombre} Precio de compra: {self.precio_compra} Precio de venta: {self.precio_venta} Stock: {self.stock}"
```

9. Mostrando todos los regiistro de categoria con el comando **> Categoria.objects.all() ,** podra notare que ya puede visualizar en consola los id

```
In [1]: from catalogo.models.producto import Producto

In [2]: from catalogo.models.categoria import Categoria

In [3]: Categoria.objects.all()
Out[3]: <QuerySet [<Categoria: id: 1 Nombre: Electrónicos Description: Productos electrónicos Fecha de creación : 2026-09-14 18:43:41.575695+00:00 Fecha de actualización 2026-09-14 18:43:41.575724+00:00>, <Categoria: id: 3 Nombre: Laptop Description: Computadora portatil Fecha de creación : 2026-09-14 21:29:19.524546+00:00 Fecha de actualización 2026-09-14 21:29:19.524621+00:00>]>

In [4]:
```

Ultimo elemento

10. Para obtener el ultimo elemento podemos utilizar el comando **>Categoria.objects.last()**

```
In [8]: Categoria.objects.last()
Out[8]: <Categoria: id: 3 Nombre: Laptop Description: Computadora portatil Fecha de creación : 2026-09-14 21:29:19.524546+00:00 Fecha de actualización 2026-09-14 21:29:19.524621+00:00>
```

Obtener un reguistro espesificico **dado el id**

11. Para poder extraer un registro dado el id podemos utilizar el comadno **> Categoria.objects.get(pk =1)**

```
In [4]: Categoria.objects.get(pk =1)
Out[4]: <Categoria: id: 1 Nombre: Electrónicos Description: Productos electrónicos Fecha de creación : 2026-09-14 18:43:41.575695+00:00 Fecha de actualización 2026-09-14 18:43:41.575724+00:00>
```

12. Cuidado con el comando anterior esto porque si el pk no esta en la base de datos , python se rompe y lanza una excepción **>Categoria.objects.get(pk =5)**

```
In [5]: Categoria.objects.get(pk =5)
---------------------------------------------------------------------------
DoesNotExist                              Traceback (most recent call last)
Cell In[5], line 1
----> 1 Categoria.objects.get(pk =5)

File ~\Documents\Proyectos\django\.venv\Lib\site-packages\django\db\models\manager.py:87, in BaseManager._get_queryset_methods.<locals>.create_method.<locals>.manager_method(self, *args, **kwargs)
     85 @wraps(method)
     86 def manager_method(self, *args, **kwargs):
---> 87     return getattr(self.get_queryset(), name)(*args, **kwargs)

File ~\Documents\Proyectos\django\.venv\Lib\site-packages\django\db\models\query.py:722, in QuerySet.get(self, *args, **kwargs)
    720     return clone._result_cache[0]
    721 if not num:
--> 722     raise self.model.DoesNotExist(
    723         "%s matching query does not exist." % self.model._meta.object_name
    724     )
    725 raise self.model.MultipleObjectsReturned(
    726     "get() returned more than one %s -- it returned %s!"
    727     % (
   (...)    730     )
    731 )

DoesNotExist: Categoria matching query does not exist.
```

Conteo de elemetos

13. Para poder contare cuantos reguisitro hay en la base de datos podemos utilizar el comando **> Categoria.objects.count()**

```
In [7]: Categoria.objects.count()
Out[7]: 2
```

Cracion de reguistros

14. Cuando tarbajamos desde la consola nos podemos olvidar de la estructura del modelo ipython que es nuestra consola interactiva nos permite visualizar esta estrucuta utilizando el siguiente simbolo **?? ,** podemo probarlo con el comanod **> ategoria??**

```
In [9]: Categoria??
Init signature: Categoria(*args, **kwargs)
Docstring:      Categoria(id, nombre, descripcion, created_at, updated_at)
Source:
class Categoria(models.Model):
    # define un campo de tipo texto con máximo de 255 caracteres
    nombre = models.TextField(max_length=255)
    # definite un campo de tipo texto que puede ser nulo con máximo de 255 caracteres
    descripcion = models.TextField(max_length=255, null=True)
    # define campos de tipo fecha y hora para almacenar la fecha de creación y actualización del registro
    created_at = models.DateTimeField()
    updated_at = models.DateTimeField()

    # function que mostrar la information de la tabla en consola
    def __str__(self):
        return f"id: {self.id} Nombre: {self.nombre} Description: {self.descripcion} Fecha de creación : {self.created_at} Fecha de actualización {self.updated_at}"
File:           ~\documents\proyectos\django\comercial\catalogo\models\categoria.py
Type:           ModelBase
Subclasses:
```

15. Anterior mente habiamos creado reguistros , pero todo se hace desde la memoria y teniamos que asegurar los cambios en la base de dato con el comando **save() ,** hoy podemos realizar lo mismo con el manejador objects , pero este no necesita utlizar el comando save , el lo hace de manera directa
16. Podemos crar una nueva categoria con el comando **> Categoria.objects.create(nombre='Electrodomesticos' , descripcion='Productos electrodomesticos', created_at='2026-09-15', updated_at='2026-09-15')**

```
In [10]: Categoria.objects.create(nombre='Electrodomesticos' , descripcion='Productos electrodomesticos', creat
       : ed_at='2026-09-15', updated_at='2026-09-15')
C:\Users\cruzr\Documents\Proyectos\django\.venv\Lib\site-packages\django\db\models\fields\__init__.py:1712: Runt
imeWarning: DateTimeField Categoria.created_at received a naive datetime (2026-09-15 00:00:00) while time zone s
upport is active.
  warnings.warn(
C:\Users\cruzr\Documents\Proyectos\django\.venv\Lib\site-packages\django\db\models\fields\__init__.py:1712: Runt
imeWarning: DateTimeField Categoria.updated_at received a naive datetime (2026-09-15 00:00:00) while time zone s
upport is active.
  warnings.warn(
Out[10]: <Categoria: id: 4 Nombre: Electrodomesticos Description: Productos electrodomesticos Fecha de creación
: 2026-09-15 Fecha de actualización 2026-09-15>
```

No anteriro envia una advertencia por la fecha , aun asi el regusitro fue agregado de manera satisfactoria , para fecha podemos enviar un texto pero recuerde el formato debera ser año-mes-dia , o puede utilizar el timezone , como se realizo en las guias anteriores

Utilizando filtros

17. Mientras get() busca un único registro, filter() puede devolver cero, uno o muchos registros , devolviendo una lista de datos
18. Podemos probar con el comando **> Categoria.objects.filter(nombre='Electrónicos')**

```
In [16]: Categoria.objects.filter(nombre='Electrónicos')
Out[16]: <QuerySet [<Categoria: id: 1 Nombre: Electrónicos Description: Productos electrónicos Fecha de creación
 : 2026-09-14 18:43:41.575695+00:00 Fecha de actualización 2026-09-14 18:43:41.575724+00:00>]>
```

19. Filter es muy poderoso y tambien lo podemos utilizar con los id , con el comando >Categoria.objects.filter(pk=1)

```
In [17]: Categoria.objects.filter(pk=1)
Out[17]: <QuerySet [<Categoria: id: 1 Nombre: Electrónicos Description: Productos electrónicos Fecha de creación
 : 2026-09-14 18:43:41.575695+00:00 Fecha de actualización 2026-09-14 18:43:41.575724+00:00>]>
```

20. Con filter tambien podemos bucar productos que su stop sea mayor a 3 con el comando _gt **, > Producto.objects.filter(stock__gt=3)**

```
In [21]: Producto.objects.filter(stock__gt=3)
Out[21]: <QuerySet [<Producto: id: 1 Nombre: Samsung Galaxy book 4 Precio de compra: 700.00 Precio de venta: 150
0.00 Stock: 5>]>
```

Otros operadores importantes son

Operador	Significado	Ejemplo
gt	Mayor que >	stock__gt=10
gte	Mayor o igual >=	stock__gte=10
lt	Menor que <	stock__lt=10
lte	Menor o igual <=	stock__lte=10
contains	Contiene texto	nombre__contains="Laptop"
icontains	Contiene ignorando mayúsculas	nombre__icontains="laptop"
startswith	Comienza con	nombre__startswith="Lap"
endswith	Termina con	nombre__endswith="USB"
range	Dentro de un rango	precio_venta__range=(10,100)

Nota :

Cuando utilizamos filter podemos utilizar mas de una sola condicional , solo tenbemos que separarlas por coma > **Producto.objects.filter(stock__gt=5, precio_venta__lt=100)**

Consultas relacionadas

21. Una de las ventajas más importantes del ORM es poder realizar consultas entre modelos relacionado , para ellos podemos utilizar un ejemplo muy basico
22. Para realizar una prurbas busquemos los productos que en su categoria sean de tipo Laptop , esto lo realizaremos con el comanod **> Producto.objects.filter(categoria__nombre='Laptop')**

```
In [25]: Producto.objects.filter(categoria__nombre='Laptop')
Out[25]: <QuerySet [<Producto: id: 1 Nombre: Samsung Galaxy book 4 Precio de compra: 700.00 Precio de venta: 150
0.00 Stock: 5>]>
```

Consultas de categoiras a productos

23. Otra de las consutlas quepodriamos realizar es desde las categoira para identicar que productos estan asociados , recuede que cuando se creao el modelo producto se realciono con la categoria , pero lo mas importante es el related_name , este los permita desde la categoira preguntar sobre los productos

```python
from .categoria import Categoria


class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(max_length=255, null=True)
    # max_digits es el número máximo de dígitos que puede tener el número, incluyendo los
    decimales
    precio_compra = models.DecimalField(max_digits=10, decimal_places=2)
    precio_venta = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField()
    # relación de uno a muchos, un producto pertenece a una categoria y una categoria puede
    tener muchos productos
    categoria = models.ForeignKey(
        Categoria, on_delete=models.CASCADE, related_name='productos')
    # también lo puede ponder de esta manera
    # stock = models.IntegerField()
    # auto_now_add agrega la fecha de manera automática cuando se crea el objeto
    created_at = models.DateTimeField(auto_now_add=True)
    # auto_now actualiza la fecha de manera automática cuando se actualiza el objeto
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"id: {self.id} Nombre: {self.nombre} Precio de compra: {self.precio_compra}
        Precio de venta: {self.precio_venta} Stock: {self.stock}"
```

24. Primero preguntemos sobre una categira en especia con el comadodo **> categoria1 = Categoria.objects.get(pk=3)**

```
In [33]: categoria1 = Categoria.objects.get(pk=3)
```

25. Hoy desde de es categoira1 preguntamos sonbre los productos con el comando > categoria1.productos.all()

```
In [34]: categoria1.productos.all()
Out[34]: <QuerySet [<Producto: id: 1 Nombre: Samsung Galaxy book 4 Precio de compra: 700.00 Precio de venta: 150
0.00 Stock: 5>]>
```

26.

	**EJERCICIOS DE PRÁCTICA:**

Ejercicio práctico 1

Utilizando únicamente Django Shell y el Manager objects, realice las siguientes operaciones , deje contancia de su trabajo generadon un documeto en formato pdf:

1. Cree una categoría llamada Accesorios.
2. Cree una categoría llamada Almacenamiento.
3. Agregue al menos cinco productos.
4. Consulte todos los productos.
5. Obtenga el producto con pk=5.
6. Busque productos cuyo stock sea mayor que 10.
7. Busque productos cuyo precio de venta sea mayor o igual a $50.
8. Busque productos cuyo nombre contenga la palabra USB.
9. Ordene los productos desde el precio más alto hasta el más bajo.
10. Muestre todos los productos pertenecientes a la categoría Accesorios.
11. Seleccione una categoría y obtenga sus productos mediante categoria.productos.all().
12. Cuente cuántos productos existen.
13. Compruebe si existen productos con stock igual a cero.

Ejercico pracito 2 :

Realiza los cambiso necesarios en en la vista de categorias para enviar las categorias almacenadas en la base de datos al template correspondiente